Krylov Subspace Methods for Linear Systems

全书翻译进度稿

本文件只收录已完成的译文单元. 尚未翻译的引用目标列于文末源文定位表, 不表示相应内容已完成.

序言

在科学计算和数据科学的许多领域, 我们经常面临求解形如 $A\boldsymbol{x} = \boldsymbol{b}$ 的大型稀疏线性方程组的问题, 这是整个计算过程中最耗时的部分之一. 因此, 许多研究者致力于开发求解线性方程组的高效数值算法. 如今, Krylov 子空间方法是广泛使用的数值算法, 被认为是 20 世纪十大算法之一, 其他算法包括快速 Fourier 变换和快速排序 [39]. 尽管基本理论在 20 世纪就已建立, 数学家, 工程师, 物理学家和许多其他领域的研究者仍不断发展着 Krylov 子空间方法. 关于 Krylov 子空间方法, 已有许多优秀著作, 包括下列作者的著作:

- Owe Axelsson, 1994 [10],
- Richard Barrett et al., 1994 [18],
- Wolfgang Hackbusch, 1994 [92],
- Are Magnus Bruaset, 1995 [29],
- Rüdiger Weiss, 1996 [203],
- Anne Greenbaum, 1997 [81],
- Yousef Saad, 2003 [151],
- Henk A. van der Vorst, 2003 [196],
- Jörg Liesen 和 Zdeněk Strakoš, 2012 [122],
- Gérard Meurant 和 Jurjen Duintjer Tebbens, 2020 [129].

文献 [129] 详细记述了 Krylov 子空间方法的历史, 是一部关于非 Hermitian 线性方程组的 Krylov 子空间方法的巨著 (约 700 页). 本书的特点如下:

(1) 线性方程组在计算科学和数据科学中的许多应用;

(2) 复对称线性方程组的 Krylov 子空间方法, 如 COCG 方法和 COCR 方法;

(3) 非 Hermitian 线性方程组的 Krylov 子空间方法, 如 BiCR 方法, GPBiCG 方法和 (块) IDR(s) 方法;

(4) 移位线性方程组的 Krylov 子空间方法, 如移位 IDR(s) 方法;

(5) 作为移位线性方程组应用的矩阵函数.

特点 (1) 对应于 Chapter 2, 其中从各种应用中导出线性方程组: 偏微分方程 (有限差分离散方法和有限元方法); 计算物理: 凝聚态物理 (Green 函数的计算) 和格点量子色动力学 (Wilson 费米子矩阵); 机器学习 (最小二乘问题); 矩阵方程 (Sylvester 型矩阵方程); 最优化 (使用张量计算记号表示 Euclidean 空间和 Riemannian 流形上的 Hessian 矩阵). 特点 (2) 和 (3) 对应于 Chapter 3. 该章将 Krylov 子空间方法分成三类: 针对 Hermitian 线性方程组, 复对称线性方程组和非 Hermitian 线性方程组的方法. 对于 Hermitian 线性方程组, 从 Lanczos 过程的矩阵形式导出 CG 方法, 从 CG 方法导出 CR 方法, 并从 Lanczos 过程导出 MINRES 方法. 对于复对称线性方程组, 分别将 COCG 方法, COCR 方法和 QMR_SYM 方法作为 CG 方法, CR 方法和 MINRES 方法的推广加以介绍. 对于非 Hermitian 线性方程组, 分别将 BiCG 方法, BiCR 方法和 QMR 方法作为 CG 方法, CR 方法和 MINRES 方法的推广加以介绍. 该章还详细推导了 GPBiCG 方法和 (块) IDR(s) 方法, 这是本书的特点之一. 此外, 还简要介绍了一些预条件技术. 特点 (4) 对应于 Chapter 4. 该章将移位线性方程组的 Krylov 子空间方法分成三类: 针对 Hermitian, 复对称和非 Hermitian 线性方程组的方法, 并系统地给出这些算法的详细推导. 特点 (5) 对应于 Chapter 5. 如果需要计算大型矩阵函数, 那么 Krylov 子空间方法以及移位线性方程组的 Krylov 子空间方法就是可选的方法, 因为这些算法能够计算矩阵函数中的任意元素. 该章还介绍矩阵函数的定义和一些著名的矩阵函数算法. 本书的另一个特点是, 除了为进一步理解 Krylov 子空间方法的收敛行为而给出的一些典型数值例子以外, 没有数值实验. Krylov 子空间方法的收敛性高度依赖于系数矩阵, 系数矩阵改变时, 最佳算法也会随之改变. 因此, 如果读者需要求解线性方程组, 我建议对自己的问题尝试若干种 Krylov 子空间方法 (包括 BiCGSTAB 方法和 GMRES 方法), 并从中选出最佳方法. 本书适合任何学过线性代数并且需要求解大型稀疏线性方程组的读者. 我希望本书能够帮助读者理解 Krylov 子空间方法的原理和性质, 并正确地使用 Krylov 子空间方法求解自己的问题.

日本名古屋　　Tomohiro Sogabe

2022 年 9 月

致谢

感谢大连理工大学的 Prof. Dr. Lei Du 慷慨地向我提供他的块 IDR(s) 方法的 LaTeX 代码. 感谢 Masayuki Nakamura 的工作. 他及时的提醒以及亲切的节日问候, 使我能够提高撰写本书的优先级. 感谢 Lokeshwaran Manickavasagam 和 Poojitha Ravichandran 在本书制作过程中提供的帮助. 感谢三位匿名审稿人仔细阅读书稿, 并提出极具建设性的意见, 提高了书稿质量. 我衷心感谢 Springer Series in Computational Mathematics (SSCM) 编委会的三位编辑: Prof. Dr. Randolph E. Bank, Prof. Dr. Wolfgang Hackbusch, Prof. Dr. Josef Stoer 和 Prof. Dr. Harry Yserentant, 他们以广博而杰出的专业视角为作者提供了富有成效的意见. 最后, 感谢 Prof. Dr. Shao-Liang Zhang, 大约 20 年前我在东京大学攻读博士学位时, 是他引领我进入 Krylov 子空间方法领域. 我还要感谢我亲爱的妻子 Yuan Li, 感谢她日复一日的支持与关爱. 她建议我利用周末撰写本书, 同时照料两个活泼的儿子 Kazuma 和 Takuma, 以及她年迈的父亲.

1 线性方程组数值解法导论

摘要 求解线性方程组的数值方法分为两类: 直接方法和迭代方法. 直接方法通过有限次算术运算求解线性方程组, 其中最著名的是 LU 分解. 迭代方法生成一个近似解序列, 大致分为定常迭代方法和 Krylov 子空间方法. 多重网格方法也是一类重要的迭代方法. 本章旨在介绍直接方法和定常迭代方法的原理, 并简要介绍 Krylov 子空间方法的理论. 此外, 还将简要说明多重网格方法.

1.1 线性方程组

本书考虑下列形式的线性方程组:

$$A\boldsymbol{x} = \boldsymbol{b}, \tag{1.1}$$

其中 $A \in \mathbb{C}^{N\times N}$, $\boldsymbol{b} \in \mathbb{C}$, 并假设系数矩阵 A 非奇异, 即 A 的逆存在. 设 $\tilde{\boldsymbol{x}}$ 为线性方程组 (1.1) 的一个近似解. 定量了解解 $\boldsymbol{x}$ 与近似解 $\tilde{\boldsymbol{x}}$ 之间距离的一种方法, 是计算 $\boldsymbol{x} - \tilde{\boldsymbol{x}}$ 的向量范数. 类似地, 两个矩阵 A, B 之间的距离可由 $A - B$ 的矩阵范数定量衡量. 向量范数和矩阵范数的定义将在下面两个小节中介绍. 这里的说明以 [170] 为基础; 关于数值线性代数的详细内容, 参见 [45], [79], [107], [191].

1.1.1 向量范数

向量 $\boldsymbol{x} := (x_1, x_2)^\top \in \mathbb{R}^2$ 的长度为 $(x_1^2 + x_2^2)^{1/2}$. 范数是长度概念的推广. 如果对所有 $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n, \alpha \in \mathbb{R}$, 映射 $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ 满足下列规则, 就称它为向量范数:[1]

(Nv1) $\|\boldsymbol{x}\| \geq 0$; $\|\boldsymbol{x}\| = 0 \iff \boldsymbol{x} = \boldsymbol{0}$. (正性)

(Nv2) $\|\alpha\boldsymbol{x}\| = |\alpha|\|\boldsymbol{x}\|$. (齐次性)

(Nv3) $\|\boldsymbol{x} + \boldsymbol{y}\| \leq \|\boldsymbol{x}\| + \|\boldsymbol{y}\|$. (三角不等式)

例如, p-范数 ($p \geq 1$) 定义为

$$\|\boldsymbol{x}\|_p := (|x_1|^p + |x_2|^p + \cdots + |x_n|^p)^{\frac{1}{p}}.$$

当 $p = 1, 2, \infty$ 时, 向量 p-范数写为

- $\|\boldsymbol{x}\|_1 = \sum_{i=1}^{n} |x_i|$,
- $\|\boldsymbol{x}\|_2 = (\sum_{i=1}^{n} |x_i|^2)^{\frac{1}{2}}$ $(= \sqrt{\boldsymbol{x}^\top \boldsymbol{x}})$,
- $\|\boldsymbol{x}\|_\infty = \max_{1\leq i\leq n} |x_i|$.

设 A 为对称正定矩阵, 即 A 对称且其全部特征值均为正. 则称 $\|\boldsymbol{x}\|_A := \boldsymbol{x}^\top A\boldsymbol{x}$ 为 $\boldsymbol{x}$ 的 A-范数; Section 3.1.1 将使用这一范数. 设 $\tilde{\boldsymbol{x}}$ 为线性方程组 (1.1) 的一个近似解. 则 $\|\boldsymbol{x} - \tilde{\boldsymbol{x}}\|_p$ 是 (1.1) 的精确解与近似解之间的距离. 本书中除非另有说明, 符号 $\|\cdot\|$ 都表示 2-范数, 即 $\|\cdot\| = \|\cdot\|_2$.

[1] 一般地, 给定向量空间 V, 如果映射 $\|\cdot\| : V \to \mathbb{R}$ 对所有 $\boldsymbol{x}, \boldsymbol{y} \in V, \alpha \in \mathbb{K}$ ($\mathbb{K} = \mathbb{R}$ 或 $\mathbb{C}$) 满足 (Nv1)–(Nv3), 就称它为范数.

1.1.2 矩阵范数

类似于向量范数的定义, 矩阵范数定义如下: 如果映射 $\|\cdot\|:\mathbb{R}^{m\times n}\to\mathbb{R}$ 对所有 $A,B\in\mathbb{R}^{m\times n},\alpha\in\mathbb{R}$ 满足下列三项, 就称它为矩阵范数:

(Nm1) $\|A\|\geq 0;\quad \|A\|=0\iff A=O,$ (O: 零矩阵) (正性)

(Nm2) $\|\alpha A\|=|\alpha|\|A\|,$ (齐次性)

(Nm3) $\|A+B\|\leq\|A\|+\|B\|.$ (三角不等式)

此外, 若矩阵范数满足

(Nm4) $\|AB\|\leq\|A\|\|B\|,\quad A\in\mathbb{R}^{m\times n},B\in\mathbb{R}^{n\times q},$

则称该矩阵范数具有次乘性. 注意, 有些著作也将满足 (Nm1)–(Nm4) 的映射称为矩阵范数. 作为具有次乘性的矩阵范数的一个例子, 下列从属矩阵范数被广泛使用:[2]

$$\|A\|_p:=\sup_{\boldsymbol{x}\neq\boldsymbol{0}}\frac{\|A\boldsymbol{x}\|_p}{\|\boldsymbol{x}\|_p}. \tag{1.2}$$

当 $p=1,2,\infty$ 时, $\|A\|_p$ 如下:

- $\|A\|_1=\max\limits_j\sum\limits_{i=1}^{n}|a_{ij}|,$
- $\|A\|_2=\sqrt{\text{maximum eigenvalue of } A^\top A},$
- $\|A\|_\infty=\max\limits_i\sum\limits_{j=1}^{n}|a_{ij}|.$

Frobenius 范数也具有次乘性:

- $\|A\|_{\mathrm{F}}:=\sqrt{\sum\limits_{i=1}^{m}\sum\limits_{j=1}^{n}|a_{ij}|^2}.$

设 $\boldsymbol{a}_1,\ldots,\boldsymbol{a}_n$ 为矩阵 $A\in\mathbb{R}^{m\times n}$ 的列向量, 并令 $\boldsymbol{a}=[\boldsymbol{a}_1^\top,\ldots,\boldsymbol{a}_n^\top]^\top$, 即将全部列首尾相接得到一个长向量 $\boldsymbol{a}$. 则 A 的 Frobenius 范数等于 $\boldsymbol{a}$ 的 2-范数, 即 $\|A\|_{\mathrm{F}}=\|\boldsymbol{a}\|_2$. 下列形式的最大值范数

$$\|A\|_{\max}:=\max_{i,j}|a_{ij}|$$

不具有次乘性. 例如, 考虑所有元素均为 1 的 2 行 2 列矩阵 A. 则 $\|AA\|_{\max}=\|A^2\|_{\max}=2$, 而 $\|A\|_{\max}\|A\|_{\max}=1$. 因此, 最大值范数不满足 (Nm4), 因为 $\|AA\|_{\max}>\|A\|_{\max}\|A\|_{\max}$. 下面给出矩阵范数的一些性质.

Theorem 1.1:

下列性质成立:

(1) $\|A\boldsymbol{x}\|_p\leq\|A\|_p\|\boldsymbol{x}\|_p,\quad \|A\boldsymbol{x}\|_2\leq\|A\|_{\mathrm{F}}\|\boldsymbol{x}\|_2,$

[2]Sup 可以替换为 max.

(2) $\|AB\|_p \le \|A\|_p\|B\|_p, \quad \|AB\|_{\mathrm{F}} \le \|A\|_{\mathrm{F}}\|B\|_{\mathrm{F}},$

(3) $\|QA\tilde{Q}\|_2 = \|A\|_2, \quad \|QA\tilde{Q}\|_{\mathrm{F}} = \|A\|_{\mathrm{F}}.$

这里, Q 和 $\tilde{Q}$ 是正交矩阵.[a]

[a]若实方阵 Q 满足 $Q^\top Q = I$, 则称其为正交矩阵, 其中 I 是单位矩阵.

Theorem 1.2:

设 A 为 $n \times n$ 矩阵. 则

(1) $n^{-1/2}\|A\|_2 \le \|A\|_1 \le n^{1/2}\|A\|_2,$

(2) $n^{-1/2}\|A\|_2 \le \|A\|_\infty \le n^{1/2}\|A\|_2,$

(3) $n^{-1}\|A\|_\infty \le \|A\|_1 \le n\|A\|_\infty,$

(4) $\|A\|_1 \le \|A\|_{\mathrm{F}} \le n^{1/2}\|A\|_2.$

1.2 条件数

考虑 $A\boldsymbol{x} = \boldsymbol{b}$; 如果矩阵 A 和右端向量 $\boldsymbol{b}$ 发生微小变化, 解会改变多少? 为回答这一问题, 设 ΔA 和 $\Delta\boldsymbol{b}$ 分别为 A 和 $\boldsymbol{b}$ 的扰动. 以 $\Delta\boldsymbol{x}$ 表示的解的变化满足下列关系:

$$(A + \Delta A)(\boldsymbol{x} + \Delta\boldsymbol{x}) = \boldsymbol{b} + \Delta\boldsymbol{b}.$$

利用 $A\boldsymbol{x} = \boldsymbol{b}$ 得到

$$\Delta\boldsymbol{x} = A^{-1}[-\Delta A(\boldsymbol{x} + \Delta\boldsymbol{x}) + \Delta\boldsymbol{b}].$$

利用 Theorem 1.1-(1) 和三角不等式 (Nv3), 可由下式估计 $\boldsymbol{x}$ 的 p-范数:

$$\|\Delta\boldsymbol{x}\|_p \le \|A^{-1}\|_p(\|\Delta A\|_p\|\boldsymbol{x} + \Delta\boldsymbol{x}\|_p + \|\Delta\boldsymbol{b}\|_p).$$

因此, 有

$$\frac{\|\Delta\boldsymbol{x}\|_p}{\|\boldsymbol{x} + \Delta\boldsymbol{x}\|_p} \le \|A\|_p\|A^{-1}\|_p \left(\frac{\|\Delta A\|_p}{\|A\|_p} + \frac{\|\Delta\boldsymbol{b}\|_p}{\|A\|_p\|\boldsymbol{x} + \Delta\boldsymbol{x}\|_p}\right). \tag{1.3}$$

这里,

$$\kappa_p(A) := \|A\|_p\|A^{-1}\|_p \tag{1.4}$$

称为 p-范数条件数. 不等式 (1.3) 表明, 如果条件数很大, 计算得到的近似解可能远离精确解. 注意, 这一事实与求解线性方程组所使用的算法无关. 如果条件数很大, 则称该线性方程组是病态的. 我们将在 Chapter 3 看到, 条件数也会影响 Krylov 子空间方法的收敛速度. 本书经常使用 2-范数条件数 $\kappa_2(A)$, 并将其简记为 κ 或 $\mathrm{cond}(A)$.

1.3 直接方法

为简化叙述, 仅考虑 $A\boldsymbol{x} = \boldsymbol{b}$, 其中 $A \in \mathbb{R}^{n\times n}$, $\boldsymbol{b} \in \mathbb{R}^n$. Section 1.3.1 介绍用 LU 分解求解线性方程组, 随后的 Section 1.3.2 说明带选主元技术的 LU 分解, 这种方法数值稳定, 在实践中广泛使用. Section 1.3.3 介绍提高近似解精度的一种方法.

1.3.1 LU 分解

设 A 为方阵. 将 A 分解为下三角矩阵 L 与上三角矩阵 U 的乘积, 就得到 A 的 LU 分解, 其形式为

$$A = LU, \tag{1.5}$$

其中

$$L = \begin{pmatrix} l_{11} & & \\ \vdots & \ddots & \\ l_{n1} & \cdots & l_{nn} \end{pmatrix}, \quad U = \begin{pmatrix} u_{11} & \cdots & u_{1n} \\ & \ddots & \vdots \\ & & u_{nn} \end{pmatrix}.$$

矩阵 L 和 U 中的空白表示相应元素均为零. 若 L 或 U 的对角元素均为 1, 则 LU 分解唯一确定. 对所有 i 都有 $l_{ii} = 1$ 的 LU 分解称为 Doolittle 方法, 对所有 i 都有 $u_{ii} = 1$ 的 LU 分解称为 Crout 方法. 一旦得到 A 的 LU 分解, 就容易通过下列过程得到线性方程组的解 $\boldsymbol{x}$:

$$A = LU, \quad L\boldsymbol{y} = \boldsymbol{b}, \quad U\boldsymbol{x} = \boldsymbol{y}. \tag{1.6}$$

具体地, 前代过程

$$y_i = \frac{1}{l_{ii}} \left(b_i - \sum_{j=1}^{i-1} l_{ij} y_j \right) \quad (i = 1, \ldots, n)$$

给出 $\boldsymbol{y}$, 随后的回代过程

$$x_i = \frac{1}{u_{ii}} \left(y_i - \sum_{j=i+1}^{n} u_{ij} x_j \right) \quad (i = n, \ldots, 1)$$

给出解 $\boldsymbol{x}$, 其中 $y_1 = b_1/l_{11}$, $x_n = y_n/u_{nn}$. LU 分解存在的充要条件是全部顺序主子式[3] 均不为零. 下面列出 LU 分解存在的几种情形.

1. 严格对角占优矩阵: $|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad (1 \leq i \leq n)$.
2. M-矩阵: $a_{ij} \leq 0$ $(i \neq j)$, 且 A^{-1} 的全部元素均非负.
3. 矩阵 $A \in \mathbb{R}^{n \times n}$ 的对称部分正定. 对称部分指 $(A + A^\top)/2$, 正定指对任意非零向量 $\boldsymbol{v}$, 均有 $\boldsymbol{v}^\top A \boldsymbol{v} > 0$. 对于任意方阵 A, 矩阵 A 都可写成对称部分 $(A + A^\top)/2$ 与斜对称部分 $(A - A^\top)/2$ 之和.

LU 分解 (Doolittle 方法) 的算法见 Algorithm 1.1, 它生成满足 $A = LU$ 的 L 和 U.

[3]使用 MATLAB 记号, A 的全部顺序主子式定义为 $\det(A(1:1,1:1))$, $\det(A(1:2,1:2))$, ..., $\det(A(1:n,1:n))$.

Algorithm 1.1 LU 分解

Input: 一个 $n \times n$ 矩阵 A

Output: 单位下三角矩阵 L 和上三角矩阵 U

1: **for** $i = 1, \ldots, n$ **do**
2: $\quad a_{ji} = a_{ji}/a_{ii} \quad (j = i+1, \ldots, n)$
3: $\quad$ **for** $j = i+1, \ldots, n$ **do**
4: $\quad\quad a_{jk} = a_{jk} - a_{ji} \times a_{ik} \quad (k = i+1, \ldots, n)$
5: $\quad$ **end for**
6: **end for**
7: $l_{ii} = 1 \quad (i = 1, \ldots, n)$
8: $l_{ij} = a_{ij} \quad (i > j)$
9: $u_{ij} = a_{ij} \quad (i \leq j)$

1.3.2 选主元的 LU 分解

对于某些 A, LU 分解可能不存在. 例如, 考虑

$$A = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}.$$

容易看出, LU 分解 (Algorithm 1.1) 会因 $a_{11} = 0$ 而发生零除并失败. 另一方面, 若交换第一行和第二行, 即

$$PA = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix} = \begin{pmatrix} 2 & 3 \\ 0 & 1 \end{pmatrix},$$

则 LU 分解不会失败, 并得到下列形式的 LU 分解

$$PA = LU,$$

其中

$$L = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 3 \\ 0 & 1 \end{pmatrix}.$$

这一技术称为选主元. 部分选主元的 LU 分解见 Algorithm 1.2. 这里, 部分选主元一词对应于 Algorithm 1.2 的第 2–6 行.

Algorithm 1.2 部分选主元的 LU 分解

Input: 一个 $n \times n$ 矩阵 A 和 $n \times n$ 单位矩阵 $P(= I)$

Output: 单位下三角矩阵 L 和上三角矩阵 U

Output: 置换矩阵 P

1: **for** $i = 1, \ldots, n$ **do**
2: 从 $\{|a_{ii}|, \ldots, |a_{ni}|\}$ 中找出最大元素 $a_{p_i i}$.
3: 对 $j = 1, \ldots, n$, 交换 a_{ij} 和 $a_{p_i j}$.
4: **if** $i \neq n$ **then**
5: 交换矩阵 P 的第 i 行和第 p_i 行.
6: **end if**
7: $a_{ji} = a_{ji}/a_{ii} \quad (j = i+1, \ldots, n)$
8: **for** $j = i+1, \ldots, n$ **do**
9: $a_{jk} = a_{jk} - a_{ji} \times a_{ik} \quad (k = i+1, \ldots, n)$
10: **end for**
11: **end for**
12: $l_{ii} = 1 \quad (i = 1, \ldots, n)$
13: $l_{ij} = a_{ij} \quad (i > j)$
14: $u_{ij} = a_{ij} \quad (i \leq j)$

Algorithm 1.2 生成 L, U 和置换矩阵 P, 使得

$$PA = LU.$$

与 LU 分解不同, 部分选主元的 LU 分解不会发生中断 (零除). 接下来, 完全选主元的 LU 分解见 Algorithm 1.3. 完全选主元一词对应于 Algorithm 1.3 的第 2–8 行. 注意, 在第 i 步, 部分选主元从 $\{|a_{k,i}| : k \in \{i, \ldots, n\}\}$ 中找最大值, 而完全选主元从 $\{|a_{k,l}| : k, l \in \{i, \ldots, n\}\}$ 中找最大值. Algorithm 1.3 生成分解因子矩阵 L 和 U 以及置换矩阵 P 和 Q, 使得

$$PAQ = LU.$$

通常使用部分选主元的 LU 分解; 如果分解精度不令人满意, 则可使用完全选主元.

Algorithm 1.3 完全选主元的 LU 分解

Input: 一个 $n \times n$ 矩阵 A 和 $n \times n$ 单位矩阵 $P(=I), Q(=I)$

Output: 单位下三角矩阵 L 和上三角矩阵 U

Output: 置换矩阵 P, Q

for $i = 1, \ldots, n$ **do**
  从 $\{|a_{kl}| : i \le k \le n, i \le l \le n\}$ 中找出最大元素 $a_{p_i q_i}$.
  对 $j = 1, \ldots, n$, 交换 a_{ij} 和 $a_{p_i j}$.
  对 $j = 1, \ldots, n$, 交换 a_{ji} 和 $a_{j q_i}$.
  if $i \neq n$ **then**
    交换矩阵 P 的第 i 行和第 p_i 行.
    交换矩阵 P 的第 i 列和第 q_i 行.
  end if
  $a_{ji} = a_{ji}/a_{ii} \quad (j = i+1, \ldots, n)$
  for $j = i+1, \ldots, n$ **do**
    $a_{jk} = a_{jk} - a_{ji} \times a_{ik} \quad (k = i+1, \ldots, n)$
  end for
end for
$l_{ii} = 1 \quad (i = 1, \ldots, n)$
$l_{ij} = a_{ij} \quad (i > j)$
$u_{ij} = a_{ij} \quad (i \le j)$

如果需要求解下列一组线性方程组

$$A\boldsymbol{x}_i = \boldsymbol{b}_i, \quad i = 1, 2, \ldots, m,$$

那么 LU 分解很有吸引力, 因为一旦得到 L 和 U, 就只需进行前代和回代, 参见 (1.6) 之后的说明.

1.3.3 迭代改进

本小节考虑用 LU 分解得到的线性方程组 (1.1) 的近似解 $\tilde{\boldsymbol{x}}$ 精度不足的情形. 此时, 如果知道使得 $\boldsymbol{x} = \tilde{\boldsymbol{x}} + \Delta\boldsymbol{x}$ 的修正向量 $\Delta\boldsymbol{x}$, 就能得到精确解 $\boldsymbol{x}$. 容易看出, 修正向量 $\Delta\boldsymbol{x}$ 对应于下列线性方程组的解:

$$A\Delta\boldsymbol{x} = \boldsymbol{r}, \tag{1.7}$$

其中 $\boldsymbol{r} := \boldsymbol{b} - A\tilde{\boldsymbol{x}}$. 求解 (1.7), 得到 (1.7) 的近似解 $\Delta\tilde{\boldsymbol{x}}$ 后, 预期修正后的近似解 $\tilde{\boldsymbol{x}} + \Delta\tilde{\boldsymbol{x}}$ 比 $\tilde{\boldsymbol{x}}$ 更精确. 这一过程可重复进行, 直到达到所需精度, 称为迭代改进. 迭代改进算法见 Algorithm 1.4.

Algorithm 1.4 $A\boldsymbol{x}=\boldsymbol{b}$ 的迭代改进

Input: $A, \boldsymbol{b}$, $A\boldsymbol{x}=\boldsymbol{b}$ 的近似解 $\tilde{\boldsymbol{x}}$

Output: 改进后的近似解 $\tilde{\boldsymbol{x}}$

1: 使用双倍精度运算计算 $\boldsymbol{r}=\boldsymbol{b}-A\tilde{\boldsymbol{x}}$.

2: 求解 $A\Delta\boldsymbol{x}=\boldsymbol{r}$, 得到近似解 $\Delta\tilde{\boldsymbol{x}}$.

3: 更新 $\tilde{\boldsymbol{x}}=\tilde{\boldsymbol{x}}+\Delta\tilde{\boldsymbol{x}}$.

4: 重复步骤 1–3, 直到达到所需精度.

若使用 LU 分解求解 $A\boldsymbol{x}=\boldsymbol{b}$, 则很容易得到近似解 $\Delta\tilde{\boldsymbol{x}}$, 因为可以重用分解因子 L 和 U, 因而只需进行前代和回代. 注意, Algorithm 1.4 第 1 步中的双倍精度运算是指: 若使用单精度运算, 则用双精度运算计算 $\boldsymbol{r}=\boldsymbol{b}-A\tilde{\boldsymbol{x}}$; 若使用双精度运算, 则用四倍精度运算计算它. 令 $\boldsymbol{f}(\boldsymbol{x}):=\boldsymbol{b}-A\boldsymbol{x}$, 并对 $\boldsymbol{f}(\boldsymbol{x})=\boldsymbol{0}$ 应用 Newton 方法, 就可将 Algorithm 1.4 解释为 Newton 方法.

1.4 对称线性方程组的直接方法

当矩阵 A 对称时, Cholesky 分解和 $LDL^{\top}$ 分解是 LU 分解的两种变形, 都是高效的直接方法. 作为准备, 先定义正定的概念:

1. 正定

   如果对所有 $\boldsymbol{x}\neq\boldsymbol{0}$ 都有 $\boldsymbol{x}^{\top}A\boldsymbol{x}>0$, 则称矩阵 A 正定.

2. 对称正定

   如果正定矩阵 A 对称, 则称矩阵 A 对称正定.

已知矩阵 A 对称正定, 当且仅当对称矩阵 A 的全部特征值均为正.

1.4.1 Cholesky 分解

如果矩阵 A 对称正定, 则存在下三角矩阵 L, 使得

$$A=LL^{\top}.$$

这一分解称为 Cholesky 分解. Cholesky 分解的算法见 Algorithm 1.5.

Algorithm 1.5 Cholesky 分解

Input: A

Output: L

1: **for** $i=1,\ldots,n$ **do**

2: $\quad l_{ii}=(a_{ii}-\sum_{k=1}^{i-1}l_{ik}^2)^{1/2}$

3: $\quad$ **for** $j=i+1,\ldots,n$ **do**

4: $\quad\quad l_{ji}=(a_{ji}-\sum_{k=1}^{i-1}l_{jk}\times l_{ik})/l_{ii}$

5: $\quad$ **end for**

6: **end for**

由于利用了对称性, Cholesky 分解的计算量约为 LU 分解的一半. 已知由 A 的正定性可知, Cholesky 分解不会发生中断, 也不需要 Section 1.3.2 所述的选主元技术.

1.4.2 $LDL^\top$ 分解

当对称矩阵 A 非正定时, 不能使用 Cholesky 分解. 另一方面, 如果全部顺序主子式均非负, 则存在对角矩阵 D (即全部非对角元素均为零) 和单位下三角矩阵 L (即全部对角元素均为 1 的下三角矩阵), 使得

$$A = LDL^\top.$$

这一分解称为 $LDL^\top$ 分解 (或修正 Cholesky 分解), 算法见 Algorithm 1.6.

Algorithm 1.6 $LDL^\top$ 分解

Input: A

Output: L, D

1: **for** $i = 2, \ldots, n$ **do**
2: **for** $j = 1, \ldots, i-1$ **do**
3: $l_{ij} = (a_{ij} - \sum_{k=1}^{j-1} l_{ik} \times l_{jk} \times d_{kk})/d_{jj}$
4: **end for**
5: $d_{ii} = a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2 \times d_{kk}$
6: **end for**

Algorithm 1.6 生成对角矩阵 D 的对角元素, 以及对所有 i 均满足 $l_{ii} = 1$ 的下三角矩阵 L 的元素 l_{ij}. 与 Cholesky 分解不同, $LDL^\top$ 分解中不出现平方根. 令 S 为非奇异矩阵, 则由 Sylvester 惯性定律可知, 对称矩阵 A 的正/零/负特征值个数, 分别与 $SAS^\top$ 的正/零/负特征值个数相同. 这说明 $LDL^\top$ 分解不仅可用于求解线性方程组, 还可以给出对称矩阵 A 的特征值的粗略分布. 因此, 若有 $A = LDL^\top$, 则正/零/负对角元素的个数就给出了 A 的正/零/负特征值的个数. 进一步, 通过计算 $A - \sigma_1 I = L_1 D_1 L_1^\top$ 和 $A - \sigma_2 I = L_2 D_2 L_2^\top$ 的 $LDL^\top$ 分解, 就可知道给定闭区间 $[\sigma_1, \sigma_2]$ 上的特征值个数. 具体地, 设 $n_1 (\geq 0)$ 为 D_1 的非负对角元素个数, $n_2^{(>0)}$ 为 D_2 的正对角元素个数. 则闭区间 $[\sigma_1, \sigma_2]$ 上的特征值个数为 $n_1^{(\geq 0)} - n_2^{(>0)}$. 这一事实可用于第 k 个 (广义) 特征值问题:

$$A\boldsymbol{x} = \lambda B\boldsymbol{x}, \tag{1.8}$$

其中 A 对称, B 对称正定. 已知全部特征值均为实数, 因而可写为 $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_k \leq \cdots \leq \lambda_n$. 第 k 个 (广义) 特征值问题就是求 λ_k 及 (1.8) 的相应特征向量. 广义特征值问题 (1.8) 等价于下列移位问题:

$$(A - \sigma B)\boldsymbol{x} = (\lambda - \sigma)B\boldsymbol{x}. \tag{1.9}$$

由于 B 对称正定, B 有 Cholesky 分解, 即 $B = L_B L_B^\top$. 因此由 (1.9) 得到

$$L_B^{-1}(A - \sigma B)L_B^{-\top}\tilde{\boldsymbol{x}} = (\lambda - \sigma)\tilde{\boldsymbol{x}},$$

其中 $\tilde{\boldsymbol{x}} = L_B^\top \boldsymbol{x}$. 现在设 D 为 $(A - \sigma B)$ 的 $LDL^\top$ 分解中的对角矩阵. 则

$$L_B^{-1} L D L^\top L_B^{-\top}\tilde{\boldsymbol{x}} = (\lambda - \sigma)\tilde{\boldsymbol{x}} \iff GDG^\top\tilde{\boldsymbol{x}} = (\lambda - \sigma)\tilde{\boldsymbol{x}},$$

其中 $G = L_B^{-1}L$. 这表明, 根据 Sylvester 惯性定律, D 的正 (或负) 对角元素个数, 等于大于 (或小于) σ 的特征值个数. 因此, 若 D 的负对角元素个数例如为 10, 则可知道满足条件 $\lambda \leq \sigma$ 且最接近 σ 的特征值 λ 是第 10 个特征值, 即 $\lambda = \lambda_{10}$. 文献 [120] 基于这一思想并结合若干加速技术, 提出了一种求 (广义) 特征值问题的第 k 个特征值和特征向量的算法. 关于第 k 个特征值问题, 另见 [156].

1.5 大型稀疏线性方程组的直接方法

考虑下列矩阵:

$$A = \begin{pmatrix} 2 & -1 & -1 & -1 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}. \tag{1.10}$$

如果矩阵中的零元素相对较多, 则称该矩阵稀疏. 反之, 如果矩阵中的零元素相对较少, 则称其稠密. 对 (1.10) 应用 LU 分解 (Algorithm 1.1), 得到

$$A = LU = \begin{pmatrix} 1 & & & \\ 1/2 & 1 & & \\ 1/2 & 1/3 & 1 & \\ 1/2 & 1/3 & 1/4 & 1 \end{pmatrix} \begin{pmatrix} 2 & -1 & -1 & -1 \\ & 3/2 & 1/2 & 1/2 \\ & & 4/3 & 1/3 \\ & & & 5/4 \end{pmatrix}.$$

对于 A 的下三角部分, 可见 $a_{32} = a_{42} = a_{43} = 0$, 但在 L 的相应下三角部分却有 $l_{32} = l_{42} = l_{43} \neq 0$. 类似地, 在 A 的上三角部分有 $a_{23} = a_{24} = a_{34} = 0$, 而在 U 的上三角部分却有 $u_{23} = u_{24} = u_{34} \neq 0$. 如果原矩阵 A 中的元素为零, 而 L 和 U 中相应的元素不为零, 则称 L 和 U 中的这些非零元素为填充元. 当矩阵 A 大型且稀疏时, 填充元可能成为计算 LU 分解的瓶颈. 对于大型稀疏矩阵, 通常只使用非零元素, 不在内存中存储零元素. 然而, LU 分解后, L 和 U 可能变得稠密, 即最坏情况下需要在内存中存储 n^2 个元素. 这说明, 若矩阵规模 n 为 $n = 1,000,000$, 并使用实数双精度运算, 则需要 $n^2 \times 8 \approx 8$ TB 内存. 若 $n = 10,000,000$, 则需要一台具有 800 TB 内存的计算机. 正如 Chapter 2 将展示的, 科学计算中产生的矩阵通常大型且稀疏. 因此, 用 LU 分解求解大型稀疏线性方程组时, 减少填充元至关重要. 下面介绍减少填充元的核心思想. 核心思想是对稀疏矩阵 A 的某些行和列作置换, 以减少填充元, 然后对置换后的矩阵作 LU 分解. 具体地, 对 PAQ 作 LU 分解, 其中 P 和 Q 是预先指定的置换矩阵. 随后求解

$$PAQ\boldsymbol{y} = P\boldsymbol{b}$$

从而得到解 $\boldsymbol{x} = Q\boldsymbol{y}$. 这里选择 P 和 Q, 使 L 和 U 中的填充元尽可能少. 下面给出减少 (1.10) 中 A 的填充元的例子. 考虑

$$P = Q = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}.$$

则由 (1.10) 得到

$$PAQ = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ -1 & -1 & -1 & 2 \end{pmatrix}.$$

对 PAQ 作 LU 分解, 得到

$$PAQ = LU = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 5 \end{pmatrix}.$$

幸运的是, 这种情形下没有产生填充元. 因而只需在内存中存储非零元素, 可有效利用内存. 虽然很难选择使填充元个数最少的最优置换矩阵 P 和 Q, 但已有近似优化方法. 著名的方法包括最小度方法和逆 Cuthill–McKee 方法. 关于这些方法的参考文献, 见 Section 3.5.5. 关于稀疏直接方法的详细内容, 例如可参见 [44].

1.6 定常迭代方法

定常迭代方法通过下列递推生成近似解:

$$\boldsymbol{x}_{k+1} = \boldsymbol{f}(\boldsymbol{x}_k), \quad k = 0, 1, \ldots \tag{1.11}$$

其中 $\boldsymbol{x}_0$ 是为 $A\boldsymbol{x} = \boldsymbol{b}$ 预先给定的初始猜测. 这里, 向量值函数 $\boldsymbol{f}$ 定义为

$$\boldsymbol{f}(\boldsymbol{y}) := M^{-1}N\boldsymbol{y} + M^{-1}\boldsymbol{b}, \tag{1.12}$$

其中 M 为非奇异矩阵, 矩阵 N 使 (1.1) 中的矩阵 A 满足 $A = M - N$. 解 $\boldsymbol{x}$ 对应于 $\boldsymbol{f}$ 的不动点, 即 $\boldsymbol{x} = \boldsymbol{f}(\boldsymbol{x})$. 因此, 定常迭代方法可看作通过 (1.11) 中的递推寻找 $\boldsymbol{f}$ 的不动点的迭代方法. 由于 $\boldsymbol{f}$ 不依赖于迭代次数 k, 所以称这些迭代方法为定常的. 由 (1.11) 和 (1.12), 迭代向量 $\boldsymbol{x}_{k+1}$ 可写为

$$\boldsymbol{x}_{k+1} = M^{-1}N\boldsymbol{x}_k + M^{-1}\boldsymbol{b} \tag{1.13}$$

其中 $k = 0, 1, \ldots$. 定常迭代方法可用于求解大型稀疏线性方程组, 因为与 LU 分解等直接方法不同, 计算近似解时不必考虑填充元, 只需在内存中存储系数矩阵的非零元素和一些工作向量. 下面介绍最著名的定常迭代方法, 即 Jacobi 方法, Gauss–Seidel (GS) 方法和逐次超松弛 (SOR) 方法. 设 $A = (a_{ij})$ 为 $A\boldsymbol{x} = \boldsymbol{b}$ 的系数矩阵, 并设

$$\begin{aligned} L &= (a_{ij}), \quad a_{ij} = 0 \quad (i \le j), \\ D &= (a_{ij}), \quad a_{ij} = 0 \quad (i \ne j), \\ U &= (a_{ij}), \quad a_{ij} = 0 \quad (i \ge j). \end{aligned}$$

则有

$$A = L + D + U.$$

L, D 和 U 分别称为严格下三角矩阵, 对角矩阵和严格上三角矩阵. 用 L, D 和 U 选择 M 与 N, 就得到 Jacobi, GS 和 SOR 方法. Table 1.1 列出了这些代表性定常迭代方法的选择.

表 1.1: 三种定常迭代方法的分类

	M	N
Jacobi 方法	D	$-(L+U)$
Gauss–Seidel 方法	$D+L$	$-U$
SOR 方法	$(D+\omega L)/\omega$	$N=[(1-\omega)D-\omega U]/\omega$

1.6.1 Jacobi 方法

由 (1.13) 和 Table 1.1, Jacobi 方法的迭代向量 $\boldsymbol{x}_k$ 如下:

$$\boldsymbol{x}_{k+1}=-D^{-1}(L+U)\boldsymbol{x}_k+D^{-1}\boldsymbol{b}.$$

实际计算中, $\boldsymbol{x}_{k+1}$ 由下列递推关系计算:

$$x_i^{(k+1)}=\frac{1}{a_{ii}}\left(b_i-\sum_{j<i}a_{ij}x_j^{(k)}-\sum_{j>i}a_{ij}x_j^{(k)}\right) \tag{1.14}$$

其中 $i=1,2,\ldots,n$, $x_i^{(k)}$ 和 b_i 分别为 $\boldsymbol{x}_k$ 和 $\boldsymbol{b}$ 的第 i 个元素, 符号 $\sum_{j<i}$ 和 $\sum_{j>i}$ 分别表示 $\sum_{j=1}^{i-1}$ 和 $\sum_{j=i+1}^{n}$.

1.6.2 Gauss–Seidel 方法

由 (1.13) 和 Table 1.1, Gauss–Seidel (GS) 方法的迭代向量 $\boldsymbol{x}_k$ 如下:

$$\boldsymbol{x}_{k+1}=-(D+L)^{-1}U\boldsymbol{x}_k+(D+L)^{-1}\boldsymbol{b}.$$

下面推导 GS 方法. 重新考虑 Jacobi 方法. 用 (1.14) 计算 Jacobi 方法的 $x_i^{(k+1)}$ 时, $x_1^{(k+1)},\ldots,x_{i-1}^{(k+1)}$ 已经计算出来, 可看作新的信息. 因此, 若用新信息 $\sum_{j<i}a_{ij}x_j^{(k+1)}$ 代替旧信息 $\sum_{j<i}a_{ij}x_j^{(k)}$, 就可期望更快的收敛. 这正是 Gauss–Seidel 方法的核心思想, 其形式如下:

$$x_i^{(k+1)}=\frac{1}{a_{ii}}\left(b_i-\sum_{j<i}a_{ij}x_j^{(k+1)}-\sum_{j>i}a_{ij}x_j^{(k)}\right) \tag{1.15}$$

其中 $i=1,2,\ldots,n$.

1.6.3 SOR 方法

由 (1.13) 和 Table 1.1, SOR 方法的迭代向量 $\boldsymbol{x}_k$ 如下:

$$\begin{aligned}\boldsymbol{x}_{k+1}&=\left[\frac{1}{\omega}(D+\omega L)\right]^{-1}\left\{\frac{1}{\omega}[(1-\omega)D-\omega U]\right\}\boldsymbol{x}_k+\left[\frac{1}{\omega}(D+\omega L)\right]^{-1}\boldsymbol{b}\\&=(D+\omega L)^{-1}[(1-\omega)D-\omega U]\boldsymbol{x}_k+(D+\omega L)^{-1}\omega\boldsymbol{b},\end{aligned}$$

等价地, SOR 方法的 $\boldsymbol{x}_{k+1}$ 可通过求解下列线性方程组得到:

$$(D+\omega L)\boldsymbol{x}_{k+1}=[(1-\omega)D-\omega U]\boldsymbol{x}_k+\omega\boldsymbol{b}.$$

SOR 方法逐元素的迭代如下:

$$\tilde{x}_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}\right),$$
$$x_i^{(k+1)} = x_i^{(k)} + \omega(\tilde{x}_i^{(k+1)} - x_i^{(k)})$$

其中 $i = 1, 2, \ldots, n$. 由此容易看出, SOR 方法是 Gauss–Seidel 方法的单参数推广, 因为第一个等式的右端与 Gauss–Seidel 方法 (1.15) 相同. 若 $\omega = 1$, SOR 方法就退化为 Gauss–Seidel 方法. 已知收敛的必要条件是 $0 < \omega < 2$. 当 A 对称时, 对称 SOR (SSOR) 方法也很著名, 因为它可用作求解对称线性方程组的 Krylov 子空间方法的预条件子. 关于 SSOR 预条件子, 见 Section 3.5.4. SSOR 方法的迭代如下:

$$(D + \omega L)\tilde{\boldsymbol{x}}_k = [(1-\omega)D - \omega U]\boldsymbol{x}_k + \omega\boldsymbol{b},$$
$$(D + \omega U)\boldsymbol{x}_k = [(1-\omega)D - \omega L]\tilde{\boldsymbol{x}}_k + \omega\boldsymbol{b}.$$

对于 SSOR 方法, (1.13) 中的 M 和 N 为

$$M_{\mathrm{SSOR}} = \frac{1}{\omega(2-\omega)}(D + \omega L)D^{-1}(D + \omega U), \tag{1.16}$$

$$N_{\mathrm{SSOR}} = \frac{1}{\omega(2-\omega)}[(1-\omega)D - \omega L]D^{-1}[(1-\omega)D - \omega U]. \tag{1.17}$$

由于 A 对称, 有 $U = L^\top$. 因此, 由 (1.16) 和 (1.17) 可知, M_{SSOR} 和 N_{SSOR} 对称. 文献 [93] 总结了包括 SSOR 方法在内的许多 SOR 方法基本结果. 关于 SOR 方法的近期发展, 例如可参见 [128], [130], [131] 及其中的参考文献. 文献 [131] 基于 SOR 方法与 [130] 中梯度系统的离散梯度方法之间的一种新联系, 提出了自适应 SOR 方法, 从而给出了 SOR 方法的几何视角.

1.6.4 定常迭代方法的收敛性

定常迭代方法的一个显著性质是, 可通过计算 $\|M^{-1}N\|$ 来估计误差的衰减率, 其中 $\|\cdot\|$ 是从属矩阵范数 (1.2). 为说明这一点, 由 (1.13) 以及关系 $A\boldsymbol{x} = \boldsymbol{b} \iff \boldsymbol{x} = M^{-1}N\boldsymbol{x} + M^{-1}\boldsymbol{b}$, 误差为

$$\begin{aligned}\boldsymbol{e}_{k+1} &:= \boldsymbol{x} - \boldsymbol{x}_{k+1}\\ &= (M^{-1}N\boldsymbol{x} + M^{-1}\boldsymbol{b}) - (M^{-1}N\boldsymbol{x}_k + M^{-1}\boldsymbol{b})\\ &= M^{-1}N(\boldsymbol{x} - \boldsymbol{x}_k)\\ &= M^{-1}N\boldsymbol{e}_k.\end{aligned} \tag{1.18}$$

因此, 误差的范数满足

$$\|\boldsymbol{e}_{k+1}\| \le \|M^{-1}N\|\|\boldsymbol{e}_k\|.$$

误差的衰减率为 $\|M^{-1}N\|$, 这表明它线性收敛. 设 λ_i $(i = 1, \ldots, n)$ 为一个 n 行 n 列矩阵 G 的特征值, 并令 $\rho(G) = \max\{|\lambda_1|, \ldots, |\lambda_n|\}$. 则称 $\rho(G)$ 为 G 的谱半径. 由 (1.18) 和 $M^{-1}N$ 的 Jordan 分解容易看出, 对任意初始猜测均收敛的充要条件是

$$\rho(M^{-1}N) < 1. \tag{1.19}$$

下面介绍保证 (1.19) 成立的一个最著名的事实. **Definition 1.3 (正则分裂):** 若 $M^{-1} \ge O$, $N \ge O$, 即 M^{-1} 和 N 的全部元素都大于或等于零, 则称 $A = M - N$ 为正则分裂.

Theorem 1.4:
若 $A = M - N$ 为正则分裂, 则

$$\rho(M^{-1}N) < 1 \iff A^{-1} \geq O.$$

Proof: 先证明 $\rho(M^{-1}N) < 1 \implies A^{-1} \geq O$. 令 $G := M^{-1}N$, λ_i 为 G 的特征值. 则 $I - G$ 的特征值均非零 (即非奇异), 因为 $I - G$ 的特征值为 $1 - \lambda_i$, 而由假设 $\rho(G) < 1$ 可知 $|\lambda_i| < 1$. 由于 M 和 $I - G$ 非奇异, $A = M(I - G)$ 也非奇异, 即 A^{-1} 存在. 由于 $\rho(G) < 1$, 有

$$A^{-1} = (I - G)^{-1}M^{-1} = (I + G + G^2 + \cdots)M^{-1}.$$

由于假设 $A = M - N$ 为正则分裂, 所以 $G \geq O$, $M^{-1} \geq O$. 因此 $A^{-1} \geq O$. 接下来证明逆命题. 由于 $A = M - N$ 为正则分裂, 有 $M^{-1} \geq O$, $N \geq O$. 因此 $G \geq O$. 由 Perron–Frobenius 定理可知, 存在非负特征向量 $\boldsymbol{x} \geq \mathbf{0}$, 使得 $G\boldsymbol{x} = \rho(G)\boldsymbol{x}$. 容易看出, $A^{-1}N = (I - G)^{-1}G$, 因而有

$$A^{-1}N\boldsymbol{x} = (I - G)^{-1}G\boldsymbol{x} = \frac{\rho(G)}{1 - \rho(G)}\boldsymbol{x}.$$

由于 $A^{-1} \geq O$, $N \geq O$, 有 $A^{-1}N\boldsymbol{x} \geq \mathbf{0}$. 这意味着 $\rho(G)/(1 - \rho(G)) \geq 0$, 即 $(0 \leq)\rho(G) \leq 1$. 由于 $(I - G)^{-1}$ 存在, 所以 $\rho(G) \neq 1$.

■

关于定常迭代方法的更多收敛定理, 例如可参见 Varga 的优秀著作 [200]. 定常迭代方法还可用作 Section 3.5.4 中 Krylov 子空间方法的预条件子.

1.7 多重网格方法

多重网格方法是一类为求解椭圆型偏微分方程所产生的线性方程组而发展起来的迭代方法. 它利用较小线性方程组 (来自分层的较粗网格) 的若干粗略近似解, 以及最小线性方程组 (来自最粗网格) 的精确近似解所提供的信息, 求得原线性方程组的良好近似解. 为得到粗略近似解, 常使用称为光滑子的定常迭代方法 (尤其是 Gauss–Seidel 方法). 光滑子这一名称来自这样一个事实: Gauss–Seidel 方法能够快速衰减线性方程组的误差或残差中的高频分量, 而多重网格方法充分利用了这一性质. 多重网格方法的显著特点是收敛率不依赖于网格尺寸, 而定常迭代方法和 Krylov 子空间方法的收敛率依赖于网格尺寸. 后来, 多重网格方法被推广到求解其他线性方程组, 不仅包括椭圆型问题产生的方程组, 也包括其他模型产生的方程组. 多重网格方法是一个研究活跃的课题, 已有许多优秀著作, 其中一些作者为:

- Hackbusch [91],
- Wesseling [204],
- Bramble [27],
- Briggs et al. [28],
- Trottenberg et al. [193],
- Lottes [125].

Wolfgang Hackbusch 的著作不仅借助简单的一维微分方程介绍多重网格方法的基本概念, 还给出算法, 程序代码以及详细的收敛分析. James Lottes 的著作则主要讨论代数多重网格方法.

1.8 Krylov 子空间方法

Krylov 子空间方法有着悠久的历史, 可追溯至 20 世纪中叶, 其中最著名的是 Hestenes 和 Stiefel 提出的共轭梯度 (CG) 方法.[4]

Krylov 子空间方法是利用如下定义的 Krylov 子空间来求解线性方程组 (1.1) 的迭代方法.

Definition 1.5 (Krylov 子空间): 设向量 $\boldsymbol{v}, A\boldsymbol{v}, \ldots, A^{n-1}\boldsymbol{v} \in \mathbb{C}^N$ 线性无关, 则

$$\begin{aligned}\mathcal{K}_n(A, \boldsymbol{v}) &= \{c_0\boldsymbol{v} + c_1 A\boldsymbol{v} + \cdots + c_{n-1}A^{n-1}\boldsymbol{v} : c_0, c_1, \ldots, c_{n-1} \in \mathbb{C}\} \\ &\quad (= \operatorname{span}\{\boldsymbol{v}, A\boldsymbol{v}, \ldots, A^{n-1}\boldsymbol{v}\})\end{aligned} \tag{1.20}$$

称为一个 (n 维) Krylov 子空间.

$\mathcal{K}_n(A, \boldsymbol{v})$ 的维数依赖于 $\boldsymbol{v}$, 对于给定的 $\boldsymbol{v}$, 其最大维数称为 Krylov 子空间的阶, 定义如下.

Definition 1.6 (Krylov 子空间的阶): 使得

$$\dim(\mathcal{K}_n(A, \boldsymbol{v})) = \dim(\mathcal{K}_{n+1}(A, \boldsymbol{v})) \tag{1.21}$$

成立的最小数 n, 称为 A 关于 $\boldsymbol{v}$ 的阶.

考虑 $A\boldsymbol{x} = \boldsymbol{b}$, 设 m 为 A 关于 $\boldsymbol{r}_0 := \boldsymbol{b} - A\boldsymbol{x}_0$ 的阶, 其中 $\boldsymbol{x}_0$ 是 $A\boldsymbol{x} = \boldsymbol{b}$ 的解的一个初始猜测. 则

$$\dim(\mathcal{K}_m(A, \boldsymbol{r}_0)) = \dim(\mathcal{K}_{m+1}(A, \boldsymbol{r}_0)).$$

这意味着存在 $c_0, c_1, \ldots, c_{m-1}$, 使得

$$A^m\boldsymbol{r}_0 = c_0\boldsymbol{r}_0 + c_1 A\boldsymbol{r}_0 + \cdots + c_{m-1}A^{m-1}\boldsymbol{r}_0.$$

将等式乘以 A^{-1}, 得到

$$A^{m-1}\boldsymbol{r}_0 = c_0 A^{-1}\boldsymbol{r}_0 + c_1\boldsymbol{r}_0 + c_2 A\boldsymbol{r}_0 + \cdots + c_{m-1}A^{m-2}\boldsymbol{r}_0.$$

由于 $A^{-1}\boldsymbol{r}_0 = A^{-1}(\boldsymbol{b} - A\boldsymbol{x}_0) = \boldsymbol{x} - \boldsymbol{x}_0$, 当 $c_0 \neq 0$ 时, 解可以写为

$$\boldsymbol{x} = \boldsymbol{x}_0 + \frac{1}{c_0}(-c_1\boldsymbol{r}_0 - c_2 A\boldsymbol{r}_0 - \cdots - c_{m-1}A^{m-2}\boldsymbol{r}_0 + A^{m-1}\boldsymbol{r}_0)$$

令 $\boldsymbol{z}_m := (-c_1\boldsymbol{r}_0 - \cdots - c_{m-1}A^{m-2}\boldsymbol{r}_0 + A^{m-1}\boldsymbol{r}_0)/c_0$. 则 $\boldsymbol{z}_m \in \mathcal{K}_m(A, \boldsymbol{r}_0)$, 这意味着解 $\boldsymbol{x}$ 满足

$$\boldsymbol{x} = \boldsymbol{x}_0 + \boldsymbol{z}_m, \quad \boldsymbol{z}_m \in \mathcal{K}_m(A, \boldsymbol{r}_0).$$

由此可见, 解 $\boldsymbol{x}$ 属于仿射空间 $\boldsymbol{x}_0 + \mathcal{K}_m(A, \boldsymbol{r}_0)$. 这一事实自然引出在仿射空间 $\boldsymbol{x}_n \in \boldsymbol{x}_0 + \mathcal{K}_n(A, \boldsymbol{r}_0)$ 中计算近似解 $\boldsymbol{x}_n$ 的想法, 或等价地, 对于 $n = 1, 2, \ldots$,

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + \boldsymbol{z}_n, \quad \boldsymbol{z}_n \in \mathcal{K}_n(A, \boldsymbol{r}_0)$$

[4]在 CG 方法提出之后, Stiefel 于 1955 年提出了共轭残差 (CR) 方法. 他还因 Stiefel 流形而闻名, 该流形是单位圆的推广.

若恰当地确定 $\boldsymbol{z}_n$, 则这一方法经过 m 步迭代就能得到精确解, 这正是 Krylov 子空间方法的基础.

下面描述 Krylov 子空间方法的框架. 设 $\boldsymbol{x}_0 \in \mathbb{C}^N$ 为初始猜测, $\boldsymbol{r}_0 := \boldsymbol{b} - A\boldsymbol{x}_0$ 为相应的残差向量. 则 Krylov 子空间方法在下列仿射空间中生成第 n 个近似解:

Krylov 子空间条件

$$\boldsymbol{x}_n = \boldsymbol{x}_0 + \boldsymbol{z}_n, \quad \boldsymbol{z}_n \in \mathcal{K}_n(A, \boldsymbol{r}_0). \tag{1.22}$$

相应的残差向量 $\boldsymbol{r}_n$ 由下式给出:

$$\boldsymbol{r}_n := \boldsymbol{b} - A\boldsymbol{x}_n = \boldsymbol{r}_0 - A\boldsymbol{z}_n, \quad \boldsymbol{r}_n \in \mathcal{K}_{n+1}(A, \boldsymbol{r}_0). \tag{1.23}$$

由于仅利用上述条件 (1.22) 无法唯一确定 $\boldsymbol{x}_n$, 为得到唯一的近似解, 还需要施加下列条件之一:

Ritz–Galerkin 途径

$$\boldsymbol{r}_n \perp \mathcal{K}_n(A, \boldsymbol{r}_0). \tag{1.24}$$

Petrov–Galerkin 途径

$$\boldsymbol{r}_n \perp W_n \subset \mathbb{C}^N, \quad \dim(W_n) = n. \tag{1.25}$$

最小残差途径

$$\min_{\boldsymbol{x}_n \in \boldsymbol{x}_0 + \mathcal{K}_n(A, \boldsymbol{r}_0)} \|\boldsymbol{b} - A\boldsymbol{x}_n\|. \tag{1.26}$$

Petrov–Galerkin 途径包含 Ritz–Galerkin 途径和最小残差途径, 如下所述.

Proposition 1.7: 下列陈述成立:

- Ritz–Galerkin 途径 (1.24) 就是在 Petrov–Galerkin 途径 (1.25) 中选择

$$W_n = \mathcal{K}_n(A, \boldsymbol{r}_0).$$

- 最小残差途径 (1.26) 就是在 Petrov–Galerkin 途径 (1.25) 中选择

$$W_n = A\mathcal{K}_n(A, \boldsymbol{r}_0), \tag{1.27}$$

其中 $A\mathcal{K}_n(A, \boldsymbol{r}_0) := \{\sum_{k=1}^{n} c_k A\boldsymbol{v}_k : c_1, c_2, \ldots, c_n \in \mathbb{C}\}$, 而 $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$ 是 $\mathcal{K}_n(A, \boldsymbol{r}_0)$ 的基向量.

Proof: 对于 Ritz–Galerkin 途径, 结论是显然的, 因此只证明 (1.26) 与 (1.27) 的等价性. 设 $V_n = [\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n]$ 是一个 N 行 n 列的矩阵, 其列向量 $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$ 是 $\mathcal{K}_n(A, \boldsymbol{r}_0)$ 的基向量. 则 (1.22) 中的 $\boldsymbol{z}_n \in \mathcal{K}_n(A, \boldsymbol{r}_0)$ 可写为 $\mathcal{K}_n(A, \boldsymbol{r}_0) \ni \boldsymbol{z}_n = \sum_{k=1}^{n} c_k \boldsymbol{v}_k = V_n \boldsymbol{c}_n$, 其中 $\boldsymbol{c}_n = [c_1, c_2, \ldots, c_n]^\top \in \mathbb{C}^n$. 将 $\boldsymbol{z}_n = V_n \boldsymbol{c}_n$ 代入 (1.23), 得到

$$\boldsymbol{r}_n = \boldsymbol{r}_0 - AV_n \boldsymbol{c}_n. \tag{1.28}$$

注意, 若 $\boldsymbol{r}_n \perp A\boldsymbol{v}_k$ $(k = 1, 2, \ldots, n)$, 则有 $\boldsymbol{r}_n \perp A\mathcal{K}_n(A, \boldsymbol{r}_0)$, 因为容易看出, 对所有 $c_i \in \mathbb{C}$, 都有 $\boldsymbol{r}_n \perp \sum_{k=1}^{n} c_k A\boldsymbol{v}_k$. 条件 $\boldsymbol{r}_n \perp A\boldsymbol{v}_k$ $(k = 1, 2, \ldots, n)$ 等价于 $(AV_n)^{\mathrm{H}} \boldsymbol{r}_n = \boldsymbol{0}$ $(:= [0, \ldots, 0]^\top)$, 因此由 (1.28) 可得 $\boldsymbol{0} = (AV_n)^{\mathrm{H}} \boldsymbol{r}_n = (AV_n)^{\mathrm{H}} \boldsymbol{r}_0 - (AV_n)^{\mathrm{H}} AV_n \boldsymbol{c}_n$. 从而向量 $\boldsymbol{c}$ 由下式确定:

$$\boldsymbol{c}_n = [(AV_n)^{\mathrm{H}} AV_n]^{-1} (AV_n)^{\mathrm{H}} \boldsymbol{r}_0. \tag{1.29}$$

现在证明 $\boldsymbol{c}_n$ 是极小化问题 (1.26) 的解. 由于使 $\|\boldsymbol{b}-A\boldsymbol{x}_n\|^2$ 取最小值的解与 (1.26) 的解相同, 我们考虑

$$\begin{aligned}\min_{\boldsymbol{x}_n\in\boldsymbol{x}_0+\mathcal{K}_n(A,\boldsymbol{r}_0)}\|\boldsymbol{b}-A\boldsymbol{x}_n\|^2 &= \min_{\boldsymbol{c}_n\in\mathbb{C}^n}\|\boldsymbol{r}_0-AV_n\boldsymbol{c}_n\|^2\\ &= \min_{\boldsymbol{c}_n\in\mathbb{C}^n}(\boldsymbol{r}_0-AV_n\boldsymbol{c}_n)^{\mathrm{H}}(\boldsymbol{r}_0-AV_n\boldsymbol{c}_n).\end{aligned}\tag{1.30}$$

下面的分解有助于求得 (1.30) 的解:

$$\|\boldsymbol{d}-M\boldsymbol{x}\|^2=(\boldsymbol{d}-M\boldsymbol{x})^{\mathrm{H}}(\boldsymbol{d}-M\boldsymbol{x})=f(\boldsymbol{x})+c,\tag{1.31}$$

其中

$$\begin{aligned}f(\boldsymbol{x})&=(M^{\mathrm{H}}M\boldsymbol{x}-M^{\mathrm{H}}\boldsymbol{d})^{\mathrm{H}}(M^{\mathrm{H}}M)^{-1}(M^{\mathrm{H}}M\boldsymbol{x}-M^{\mathrm{H}}\boldsymbol{d}),\\ c&=-\boldsymbol{d}^{\mathrm{H}}M(M^{\mathrm{H}}M)^{-1}M^{\mathrm{H}}\boldsymbol{d}+\boldsymbol{d}^{\mathrm{H}}\boldsymbol{d}.\end{aligned}$$

若 M 列满秩, 则 $M^{\mathrm{H}}M$ 是 Hermitian 正定的,[5] 从而 $f(\boldsymbol{x})\geq 0$. 这意味着 (1.31) 的极小化可通过极小化 $f(\boldsymbol{x})$ 实现. 容易看出,

$$\boldsymbol{x}=(M^{\mathrm{H}}M)^{-1}M^{\mathrm{H}}\boldsymbol{d}\tag{1.32}$$

使得 $f(\boldsymbol{x})=0$, 因而它是 (1.31) 所对应的极小化问题的解. 令 $\boldsymbol{x}=(M^{\mathrm{H}}M)^{-1}M^{\mathrm{H}}\boldsymbol{d}$, 并取 $M=AV_n$ 和 $\boldsymbol{d}=\boldsymbol{r}_0$, 就得到与 (1.29) 相同的结果.

■

Remark 1.8: 等式 (1.31) 也可用于求解下列极小化问题:

$$\min_{\omega\in\mathbb{C}}\|\boldsymbol{a}-\omega\boldsymbol{b}\|\tag{1.33}$$

因为在 (1.32) 中取 $\boldsymbol{d}=\boldsymbol{a}$, $M=\boldsymbol{b}$, $\boldsymbol{x}=\omega$, 就得到 (1.33) 的解如下:

$$\omega=\frac{\boldsymbol{b}^{\mathrm{H}}\boldsymbol{a}}{\boldsymbol{b}^{\mathrm{H}}\boldsymbol{b}}.\tag{1.34}$$

这一结果将在 Chapter 3 的 Krylov 子空间方法中使用.

现在, 代表性的 Krylov 子空间方法与三种途径 (1.24)–(1.26) 之间的关系见 Table 1.2 和 Table 1.3.

表 1.2: Hermitian 线性方程组的 Krylov 子空间方法

Ritz–Galerkin 途径	最小残差途径
CG	MINRES, CR

表 1.3: 非 Hermitian 线性方程组的一些 Krylov 子空间方法

Ritz–Galerkin 途径	Petrov–Galerkin 途径	最小残差途径
FOM	BiCG, BiCR	GMRES, GCR

针对 Hermitian 和非 Hermitian 线性方程组的 Krylov 子空间方法之间的关系如下: BiCG 方法和 FOM 是 CG 方法的推广; GMRES 方法是 MINRES 方法的推广; BiCR 方法和 GCR 方法是 CR 方法的推广. 这些 Krylov 子空间方法的详细内容见 Chapter 3.

[5]若矩阵 $G\in\mathbb{C}^{N\times N}$ 是 Hermitian 的, 且对所有 $\boldsymbol{v}(\neq\boldsymbol{0})\in\mathbb{C}^N$ 均有 $\boldsymbol{v}^{\mathrm{H}}G\boldsymbol{v}>0$, 则称 G 为 Hermitian 正定矩阵. 等价地, Hermitian 矩阵 G 的所有特征值均为正. 由此可知, 若 G 是 Hermitian 正定的, 则 G^{-1} 也是 Hermitian 正定的. 关于正定性, 另见 Section 1.4.

1.9 Krylov 子空间的正交化方法

为满足 Section 1.8 中的 Ritz–Galerkin, Petrov–Galerkin 或最小残差途径, 需要构造 Krylov 子空间的基向量. 对于非 Hermitian 矩阵, 使用 Arnoldi 过程和双 Lanczos 过程构造 Krylov 子空间的基向量. 对于复对称矩阵, 使用复对称 Lanczos 过程; 对于 Hermitian 矩阵, 使用 Lanczos 过程. Arnoldi 过程生成 Krylov 子空间 $\mathcal{K}_n(A,\boldsymbol{v})$ 的标准正交基向量. Arnoldi 过程不会发生中断 (即零除), 但计算量随 n 增大而线性增长. 双 Lanczos 过程生成 Krylov 子空间的双正交基向量. 其计算量不随 n 增大而线性增长, 因而小于 Arnoldi 方法的计算量. 然而, 它可能发生中断, 接近中断时会导致基向量的正交性丧失. 复对称 Lanczos 过程和 Lanczos 过程都是双 Lanczos 过程的特例. 特别地, 由于 Lanczos 过程也是 Arnoldi 过程的特例, Lanczos 过程兼具 Arnoldi 过程和双 Lanczos 过程的优点, 即 Lanczos 过程生成标准正交基向量, 且计算量不随 n 增大而线性增长. 下面各小节将介绍这些方法, 并给出其算法以及将在推导 Krylov 子空间方法时使用的矩阵表示.

1.9.1 Arnoldi 过程

Arnoldi 过程生成 Krylov 子空间 $\mathcal{K}_n(A,\boldsymbol{v})$ 的正交化基向量, 即基向量 $\boldsymbol{v}_1,\boldsymbol{v}_2,\dots,\boldsymbol{v}_n$ 满足

$$(\boldsymbol{v}_i,\boldsymbol{v}_j)=\delta_{i,j},$$

其中 $(\boldsymbol{x},\boldsymbol{y}):=\boldsymbol{x}^{\mathrm{H}}\boldsymbol{y}=\sum_{i=1}^{N}\bar{x}_i y_i$ 为标准点积, $\delta_{i,j}$ 为 Kronecker 符号, 即 $i=j$ 时 $\delta_{i,j}=1$, $i\neq j$ 时 $\delta_{i,j}=0$. 为简化叙述, 考虑 $n=3$ 时生成 $\mathcal{K}_n(A,\boldsymbol{v})$ 的标准正交向量, 即由 $\operatorname{span}\{\boldsymbol{v},A\boldsymbol{v},A^2\boldsymbol{v}\}$ 生成 $\boldsymbol{v}_1,\boldsymbol{v}_2,\boldsymbol{v}_3$. 从 $\operatorname{span}\{\boldsymbol{v},A\boldsymbol{v},A^2\boldsymbol{v}\}$ 获得标准正交向量的一种方法是使用 Gram–Schmidt 过程, 按下式计算 $\boldsymbol{v}_1,\boldsymbol{v}_2$ 和 $\boldsymbol{v}_3$:

$$\begin{aligned}\boldsymbol{v}_1&=z_1\boldsymbol{v},\\ \boldsymbol{v}_2&=z_2(A\boldsymbol{v}-c_1\boldsymbol{v}_1),\\ \boldsymbol{v}_3&=z_3(A^2\boldsymbol{v}-d_1\boldsymbol{v}_1-d_2\boldsymbol{v}_2).\end{aligned}$$

这里 z_i $(i=1,2,3)$ 是使归一化条件 $\|\boldsymbol{v}_i\|=1$ 成立的标量. 确定标量 c_1, 使 $\boldsymbol{v}_1$ 和 $\boldsymbol{v}_2$ 正交, 即 $(\boldsymbol{v}_1,\boldsymbol{v}_2)=0$. 类似地, 确定标量 d_1 和 d_2, 使 $\boldsymbol{v}_3\perp\boldsymbol{v}_2$, $\boldsymbol{v}_3\perp\boldsymbol{v}_1$, 即 $(\boldsymbol{v}_1,\boldsymbol{v}_3)=(\boldsymbol{v}_2,\boldsymbol{v}_3)=0$. 然而, 已知这种途径数值不稳定, 因为当 n 较大时, $A^n\boldsymbol{v}$ 和 $A^{n-1}\boldsymbol{v}$ 之间的夹角可能接近零, 从而导致严重的相消. 实际上, 在适当条件下, $A^n\boldsymbol{v}$ 收敛于 A 的一个特征向量, 其对应特征值的模在 A 的全部特征值中最大. Arnoldi 过程对 (经典) Gram-Schmidt 过程作如下小幅修改:

$$\boldsymbol{v}_1=z_1\boldsymbol{v}, \tag{1.35}$$

$$\boldsymbol{v}_2=z_2(A\boldsymbol{v}_1-c_1\boldsymbol{v}_1), \tag{1.36}$$

$$\boldsymbol{v}_3=z_3(A\boldsymbol{v}_2-d_1\boldsymbol{v}_1-d_2\boldsymbol{v}_2). \tag{1.37}$$

与 Gram–Schmidt 过程不同, 这里不计算 $A^2\boldsymbol{v}$. 下面确定各个标量. 令 $z_1=1/(\boldsymbol{v},\boldsymbol{v})^{1/2}\in\mathbb{R}$, 等价地为 $1/\|\boldsymbol{v}\|$. 则 $\|\boldsymbol{v}_1\|=1$, 因为 $\|\boldsymbol{v}_1\|=(z_1\boldsymbol{v},z_1\boldsymbol{v})^{1/2}=(z_1^2(\boldsymbol{v},\boldsymbol{v}))^{1/2}=z_1(\boldsymbol{v},\boldsymbol{v})^{1/2}=1$. 再确定 c_1, 使 $(\boldsymbol{v}_1,\boldsymbol{v}_2)=0$, 即

$$\begin{aligned}0&=(\boldsymbol{v}_1,\boldsymbol{v}_2)\\ &=(\boldsymbol{v}_1,z_2(A\boldsymbol{v}_1-c_1\boldsymbol{v}_1))\\ &=z_2((\boldsymbol{v}_1,A\boldsymbol{v}_1)-c_1(\boldsymbol{v}_1,\boldsymbol{v}_1))=z_2((\boldsymbol{v}_1,A\boldsymbol{v}_1)-c_1).\end{aligned}$$

因此 $c_1 = (\boldsymbol{v}_1, A\boldsymbol{v}_1)$, 而 z_2 由 $\|\boldsymbol{v}_2\| = 1$ 确定, 即 $z_2 = 1/\|A\boldsymbol{v}_1 - c_1\boldsymbol{v}_1\|$. 类似地, (1.37) 中的标量 d_1, d_2 由 $(\boldsymbol{v}_1, \boldsymbol{v}_3) = (\boldsymbol{v}_2, \boldsymbol{v}_3) = 0$ 确定, z_3 由 $\|\boldsymbol{v}_3\| = 1$ 确定. 经典 Gram–Schmidt 型 Arnoldi 过程见 Algorithm 1.7; 更精确的改进 Gram–Schmidt 型 Arnoldi 过程见 Algorithm 1.8.

Algorithm 1.7 经典 Gram–Schmidt 型 Arnoldi 过程

Input: 非 Hermitian 矩阵 $A \in \mathbb{C}^{N\times N}$, 并选择满足 $\|\boldsymbol{v}_1\| = 1$ 的 $\boldsymbol{v}_1$

Output: $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N$

for $n = 1, 2, \ldots, N-1$ **do**
  $h_{i,n} = (\boldsymbol{v}_i, A\boldsymbol{v}_n), \quad i = 1, 2, \ldots, n$
  $\tilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \sum_{i=1}^{n} h_{i,n}\boldsymbol{v}_i$
  $h_{n+1,n} = \|\tilde{\boldsymbol{v}}_{n+1}\|$
  $\boldsymbol{v}_{n+1} = \tilde{\boldsymbol{v}}_{n+1}/h_{n+1,n}$
end for

Algorithm 1.8 改进 Gram–Schmidt 型 Arnoldi 过程

Input: 非 Hermitian 矩阵 $A \in \mathbb{C}^{N\times N}$, 并选择满足 $\|\boldsymbol{v}_1\| = 1$ 的 $\boldsymbol{v}_1$

Output: $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N$

for $n = 1, 2, \ldots, N-1$ **do**
  $\boldsymbol{t} = A\boldsymbol{v}_n$
  for $i = 1, 2, \ldots, n$ **do**
    $h_{i,n} = (\boldsymbol{v}_i, \boldsymbol{t})$
    $\boldsymbol{t} = \boldsymbol{t} - h_{i,n}\boldsymbol{v}_i$
  end for
  $h_{n+1,n} = \|\boldsymbol{t}\|$
  $\boldsymbol{v}_{n+1} = \boldsymbol{t}/h_{n+1,n}$
end for

Arnoldi 过程可表示为矩阵形式. 例如, 考虑递推式 (1.36) 和 (1.37) 的矩阵形式. 由 (1.36) 和 (1.37) 有

$$A\boldsymbol{v}_1 = z_2^{-1}\boldsymbol{v}_2 + c_1\boldsymbol{v}_1, \quad A\boldsymbol{v}_2 = z_3^{-1}\boldsymbol{v}_3 + d_1\boldsymbol{v}_1 + d_2\boldsymbol{v}_2,$$

或等价地

$$A[\boldsymbol{v}_1, \boldsymbol{v}_2] = [\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3] \begin{bmatrix} c_1 & d_1 \\ z_2^{-1} & d_2 \\ 0 & z_3^{-1} \end{bmatrix}.$$

注意, $A[\boldsymbol{v}_1, \boldsymbol{v}_2] = [A\boldsymbol{v}_1, A\boldsymbol{v}_2]$. 类似地, 可按下面的方式得到 Algorithm 1.7 (或 Algorithm 1.8) 的矩阵形式. 设 V_n 为 $N \times n$ 矩阵, 其列是 Algorithm 1.7 (或 Algorithm 1.8) 给出的前 n 个标准正交向量, 即

$$V_n = [\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n], \quad V_n^{\mathrm{H}} V_n = I_n,$$

设 $H_{n+1,n}$ 为 $(n+1)\times n$ Hessenberg 矩阵, 其元素在 $i>j+1$ 时满足 $h_{i,j}=0$, 并设 H_n 为 $n\times n$ Hessenberg 矩阵, 如下:

$$H_{n+1,n}=\begin{bmatrix} h_{1,1} & h_{1,2} & \cdots & h_{1,n-1} & h_{1,n} \\ h_{2,1} & h_{2,2} & \cdots & h_{2,n-1} & h_{2,n} \\ & \ddots & \ddots & \vdots & \vdots \\ & & \ddots & \ddots & \vdots \\ & & & h_{n,n-1} & h_{n,n} \\ & & & & h_{n+1,n} \end{bmatrix}$$

以及

$$H_n=\begin{bmatrix} h_{1,1} & h_{1,2} & \cdots & h_{1,n-1} & h_{1,n} \\ h_{2,1} & h_{2,2} & \cdots & h_{2,n-1} & h_{2,n} \\ & \ddots & \ddots & \vdots & \vdots \\ & & \ddots & \ddots & \vdots \\ & & & h_{n,n-1} & h_{n,n} \end{bmatrix},$$

其中 $h_{i,j}$ 是 Algorithm 1.7 (或 Algorithm 1.8) 生成的标量. 则有

$$AV_n=V_{n+1}H_{n+1,n}=V_nH_n+h_{n+1,n}\boldsymbol{v}_{n+1}\boldsymbol{e}_n^{\top}, \tag{1.38}$$

其中 $\boldsymbol{e}_n=(0,0,\ldots,0,1)^{\top}\in\mathbb{R}^n$. 由上述矩阵形式以及 $V_n^{\mathrm{H}}V_n=I_n$, 得到下列形式的关系

$$H_n=V_n^{\mathrm{H}}AV_n. \tag{1.39}$$

如果 A 是 Hermitian 的, 则由 $H_n^{\mathrm{H}}=(V_n^{\mathrm{H}}AV_n)^{\mathrm{H}}=V_n^{\mathrm{H}}A^{\mathrm{H}}V_n=V_n^{\mathrm{H}}AV_n=H_n$ 可知, Hessenberg 矩阵 H_n 是 Hermitian 的. 因此, 此时 H_n 为三对角矩阵. 这意味着, 若 A 是 Hermitian 的, Arnoldi 过程就有短递推关系, 从而得到 Lanczos 过程. Lanczos 过程的详细内容将在 Section 1.9.4 介绍.

1.9.2 双 Lanczos 过程

双 Lanczos 过程是一种利用短递推获得 $\mathcal{K}_n(A,\boldsymbol{v}_0)$ 的双正交基的算法. 尽管该过程不生成标准正交基向量, 它在以较低内存需求求解线性方程组方面发挥着重要作用. 算法见 Algorithm 1.9. 如果不发生中断, Algorithm 1.9 生成两个 Krylov 子空间的双正交基向量, 满足

$$(\boldsymbol{w}_i,\boldsymbol{v}_j)=\delta_{i,j},$$

其中 $\mathcal{K}_n(A,\boldsymbol{v}_1)=\mathrm{span}\{\boldsymbol{v}_1,\ldots,\boldsymbol{v}_n\}$, $\mathcal{K}_n(A^{\mathrm{H}},\boldsymbol{w}_1)=\mathrm{span}\{\boldsymbol{w}_1,\ldots,\boldsymbol{w}_n\}$.

Algorithm 1.9 双 Lanczos 过程

Input: 非 Hermitian 矩阵 $A \in \mathbb{C}^{N\times N}$, 并选择满足 $(\boldsymbol{v}_0, \boldsymbol{w}_0) \neq 0$ 的 $\boldsymbol{v}_0$ 和 $\boldsymbol{w}_0$

Input: $\boldsymbol{v}_1 = \boldsymbol{v}_0/\|\boldsymbol{v}_0\|, \quad \boldsymbol{w}_1 = \boldsymbol{w}_0/\overline{(\boldsymbol{w}_0, \boldsymbol{v}_1)}$

Output: $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N$ 和 $\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_N$

1: **for** $n = 1, 2, \ldots, N-1$ **do**

2: $\quad \alpha_n = (\boldsymbol{w}_n, A\boldsymbol{v}_n)$,

3: $\quad \tilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \alpha_n \boldsymbol{v}_n - \beta_{n-1}\boldsymbol{v}_{n-1}$

4: $\quad \tilde{\boldsymbol{w}}_{n+1} = A^{\mathrm{H}}\boldsymbol{w}_n - \bar{\alpha}_n \boldsymbol{w}_n - \bar{\gamma}_{n-1}\boldsymbol{w}_{n-1}$

5: $\quad \gamma_n = \|\tilde{\boldsymbol{v}}_{n+1}\|$

6: $\quad \boldsymbol{v}_{n+1} = \tilde{\boldsymbol{v}}_{n+1}/\gamma_n$

7: $\quad \beta_n = (\tilde{\boldsymbol{w}}_{n+1}, \boldsymbol{v}_{n+1})$

8: $\quad \boldsymbol{w}_{n+1} = \tilde{\boldsymbol{w}}_{n+1}/\bar{\beta}_n$

9: **end for**

与 Arnoldi 过程类似, 双 Lanczos 过程也可写成矩阵形式. 设 $T_{n+1,n}$ 为 $(n+1)\times n$ 三对角矩阵, 其元素为双 Lanczos 过程的递推系数, 即

$$T_{n+1,n} = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \gamma_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \gamma_{n-1} & \alpha_n \\ & & & \gamma_n \end{bmatrix}, \quad T_n = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \gamma_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \gamma_{n-1} & \alpha_n \end{bmatrix}.$$

则由双 Lanczos 过程有

$$AV_n = V_{n+1}T_{n+1,n} = V_nT_n + \gamma_n \boldsymbol{v}_{n+1}\boldsymbol{e}_n^{\top}, \tag{1.40}$$

$$A^{\mathrm{H}}W_n = W_{n+1}T_{n+1,n}^{\mathrm{H}} = W_nT_n^{\mathrm{H}} + \bar{\beta}_n \boldsymbol{w}_{n+1}\boldsymbol{e}_n^{\top}.$$

由上式以及 $W_n^{\mathrm{H}}V_n = I_n$, 容易得到公式 $T_n = W_n^{\mathrm{H}}AV_n$.

1.9.3 复对称 Lanczos 过程

当系数矩阵为复对称矩阵, 即 $A = A^{\top} \neq A^{\mathrm{H}}$ 时, 复对称 Lanczos 过程是双 Lanczos 过程 (Algorithm 1.9) 的特例. 若 A 复对称, 则令 $\boldsymbol{w}_0 = \bar{\boldsymbol{v}}_0$, 就容易从 Algorithm 1.9 导出该过程, 得到的算法见 Algorithm 1.10. 如果不发生中断, Algorithm 1.10 生成 Krylov 子空间的共轭正交基向量, 满足

$$(\bar{\boldsymbol{v}}_i, \boldsymbol{v}_j) = \delta_{ij},$$

其中 $\mathcal{K}_n(A, \boldsymbol{v}_1) = \mathrm{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$.

Algorithm 1.10 复对称 Lanczos 过程 (注意: $(\bar{\boldsymbol{a}}, \boldsymbol{b}) = \boldsymbol{a}^{\top}\boldsymbol{b}$)

Input: 复对称矩阵 $A \in \mathbb{C}^{N\times N}$, 并选择满足 $(\bar{\boldsymbol{v}}_1, \boldsymbol{v}_1) \neq 0$ 的 $\boldsymbol{v}_1$

Input: $\boldsymbol{v}_1 = \boldsymbol{v}_1/(\bar{\boldsymbol{v}}_1, \boldsymbol{v}_1)^{1/2}$

Output: $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N$

1: **for** $n = 1, 2, \ldots, N-1$ **do**

2: $\quad \alpha_n = (\bar{\boldsymbol{v}}_n, A\boldsymbol{v}_n)$

3: $\quad \tilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \alpha_n\boldsymbol{v}_n - \beta_{n-1}\boldsymbol{v}_{n-1}$

4: $\quad \beta_n = (\bar{\tilde{\boldsymbol{v}}}_{n+1}, \tilde{\boldsymbol{v}}_{n+1})^{1/2}$

5: $\quad \boldsymbol{v}_{n+1} = \tilde{\boldsymbol{v}}_{n+1}/\beta_n$

6: **end for**

与 Arnoldi 过程和双 Lanczos 过程类似, Algorithm 1.10 也可写成矩阵形式. 设 $T_{n+1,n}$ 为 $(n+1)\times n$ 三对角矩阵, 其元素为复对称 Lanczos 过程的递推系数, 即

$$T_{n+1,n} = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \beta_{n-1} & \alpha_n \\ & & & \beta_n \end{bmatrix}, \quad T_n = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \beta_{n-1} & \alpha_n \end{bmatrix}.$$

则由 Algorithm 1.10 有

$$AV_n = V_{n+1}T_{n+1,n} = V_nT_n + \beta_n\boldsymbol{v}_{n+1}\boldsymbol{e}_n^{\top}. \tag{1.41}$$

由上式可知, T_n 也复对称, 即 $T_n = T_n^{\top} \neq T_n^{\mathrm{H}}$.

1.9.4 Lanczos 过程

Lanczos 过程是 Arnoldi 过程在 A 为 Hermitian 矩阵时的特例. 由于 H_n 既是 Hermitian 矩阵又是 Hessenberg 矩阵, 它是三对角矩阵. 这意味着, 在 Arnoldi 过程 (Algorithm 1.7) 的内循环中, 从 1 到 n 的求和可替换为从 $n-1$ 到 n 的求和. 因此, Lanczos 过程在第 n 步只需要三项递推, 而不需要 $(n+1)$ 项递推. 由于这一出色的性质, Lanczos 过程的每一步都比 Arnoldi 过程或双 Lanczos 过程的相应步骤简洁得多. Lanczos 过程见 Algorithm 1.11.

Algorithm 1.11 Lanczos 过程

Input: Hermitian 矩阵 $A \in \mathbb{C}^{N\times N}$, 并选择满足 $\|\boldsymbol{v}_1\| = 1$ 的 $\boldsymbol{v}_1$

Input: $\beta_0 = 0, \boldsymbol{v}_0 = \boldsymbol{0}$

Output: $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_N$

1: **for** $n = 1, 2, \ldots, N-1$ **do**

2: $\quad \alpha_n = (\boldsymbol{v}_n, A\boldsymbol{v}_n)$

3: $\quad \tilde{\boldsymbol{v}}_{n+1} = A\boldsymbol{v}_n - \alpha_n\boldsymbol{v}_n - \beta_{n-1}\boldsymbol{v}_{n-1}$

4: $\quad \beta_n = (\tilde{\boldsymbol{v}}_{n+1}, \tilde{\boldsymbol{v}}_{n+1})^{1/2}$

5: $\quad \boldsymbol{v}_{n+1} = \tilde{\boldsymbol{v}}_{n+1}/\beta_n$

6: **end for**

令 $\boldsymbol{w}_0 = \boldsymbol{v}_0$, 就容易从 Arnoldi 过程或双 Lanczos 过程导出 Lanczos 过程. 若不发生中断, 上述算法生成 $\mathcal{K}_n(A, \boldsymbol{v}_1)$ 的标准正交基, 满足

$$(\boldsymbol{v}_i, \boldsymbol{v}_j) = \delta_{ij},$$

其中 $\mathcal{K}_n(A, \boldsymbol{v}_1) = \mathrm{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$. 与 Arnoldi 过程和双 Lanczos 过程类似, Lanczos 过程也可写成矩阵形式. 设 $T_{n+1,n}$ 为 $(n+1) \times n$ 三对角矩阵, 其元素为 Lanczos 过程的递推系数, 即

$$T_{n+1,n} = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \beta_{n-1} & \alpha_n \\ & & & \beta_n \end{bmatrix}, \quad T_n := \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \alpha_2 & \ddots & \\ & \ddots & \ddots & \beta_{n-1} \\ & & \beta_{n-1} & \alpha_n \end{bmatrix}.$$

则由 Lanczos 过程有

$$AV_n = V_{n+1}T_{n+1,n} = V_nT_n + \beta_n\boldsymbol{v}_{n+1}\boldsymbol{e}_n^{\top}. \tag{1.42}$$

由上式以及 $V_n^{\mathrm{H}}V_n = I_n$, 容易得到下列公式:

$$T_n = V_n^{\mathrm{H}}AV_n. \tag{1.43}$$

可见 T_n 也是 Hermitian 的 (更确切地说, 是实对称的). Lanczos 过程用于求解 Hermitian 线性方程组.

尚未翻译部分的源文定位

以下为已核实的源文引用目标. 这里只提供定位, 不是尚未完成章节的译文.

Some Applications to Computational Science and Data Science

Source 2, PDF 43, printed 31.

Classification and Theory of Krylov Subspace Methods

Source 3, PDF 85, printed 73.

Applications to Matrix Functions

Source 5, PDF 201, printed 189.

Applications to Shifted Linear Systems

Source 4, PDF 175, printed 163.

The Conjugate Gradient (CG) Method

Source 3.1.1, PDF 85, printed 73.

Reorderings for Preconditioners

Source 3.5.5, PDF 173, printed 161.

Preconditioners Based on Stationary Iterative Methods

Source 3.5.4, PDF 172, printed 160.
Reference [10], PDF 223, printed 213.
Reference [18], PDF 224, printed 214.
Reference [27], PDF 224, printed 214.
Reference [28], PDF 224, printed 214.
Reference [29], PDF 224, printed 214.
Reference [39], PDF 224, printed 214.
Reference [44], PDF 225, printed 215.
Reference [45], PDF 225, printed 215.
Reference [79], PDF 226, printed 216.
Reference [81], PDF 226, printed 216.
Reference [91], PDF 226, printed 216.
Reference [92], PDF 226, printed 216.
Reference [93], PDF 226, printed 216.
Reference [107], PDF 227, printed 217.
Reference [120], PDF 227, printed 217.
Reference [122], PDF 228, printed 218.
Reference [125], PDF 228, printed 218.
Reference [128], PDF 228, printed 218.
Reference [129], PDF 228, printed 218.

Reference [130], PDF 228, printed 218.
Reference [131], PDF 228, printed 218.
Reference [151], PDF 229, printed 219.
Reference [156], PDF 229, printed 219.
Reference [170], PDF 229, printed 219.
Reference [191], PDF 230, printed 220.
Reference [193], PDF 230, printed 220.
Reference [196], PDF 231, printed 221.
Reference [200], PDF 231, printed 221.
Reference [203], PDF 231, printed 221.
Reference [204], PDF 231, printed 221.